

Contents

Diseño del Segundo Parcial — Fundamentos de Programación	1
1. Ficha técnica	1
2. Objetivos de aprendizaje evaluados	1
3. Tabla de especificaciones	2
4. Estructura y preguntas modelo	3
5. Rúbrica general de calificación (por pregunta abierta, 5 pts)	5
6. Recomendaciones de aplicación	5

Diseño del Segundo Parcial — Fundamentos de Programación

Código del curso: 314029

Profesor: Marco Cañas

1. Ficha técnica

Aspecto	Detalle
Temas evaluados	Cap. 4 – Expresiones condicionales · Cap. 5 – Ciclos (for, while) · Cap. 6 (primera parte) – Funciones definidas con def
Corte de contenido	Hasta “funciones matemáticas con def” (parámetros posicionales, con nombre, por defecto, recursión). No incluye lambda, map/filter, ni módulos — ese es el corte que el propio cuaderno 6 señala para el segundo parcial.
Modalidad	Escrita, individual, sin computador (lápiz y papel; se permite trazar/depurar código a mano)
Duración	120 minutos
Puntaje total	100 puntos (20 preguntas × 5 puntos, distribuidas en 3 secciones)
Ponderación en el curso	Sugerido 25-30% de la nota final

2. Objetivos de aprendizaje evaluados

1. Traducir una condición matemática o lógica en una estructura if / elif / else correcta y bien indentada.
2. Combinar condiciones con and, or, not para validar reglas compuestas.

3. Diseñar y ejecutar mentalmente ciclos for y while, incluyendo ciclos anidados y control de flujo (break, continue, pass).
4. Aplicar ciclos a sumatorias, aproximaciones numéricas (bisección, cálculo de π) y recorrido/agregación de datos.
5. Definir funciones con def, distinguiendo parámetros posicionales, por palabra clave y con valor por defecto.
6. Representar objetos matemáticos (funciones por partes, sucesiones recursivas, composición de funciones) como funciones de Python.
7. Rastrear (“trace”) la ejecución de un fragmento de código y predecir su salida o detectar el error.

3. Tabla de especificaciones

Sección	Tema (cuaderno)	Nivel cognitivo dominante	N° preguntas	Puntos	%
I	Condicionales (Cap. 4)	Comprensión → aplicación	6	30	30%
II	Ciclos (Cap. 5)	Aplicación → análisis	7	35	35%
III	Funciones con def (Cap. 6, 1ª parte)	Análisis → síntesis	7	35	35%
Total			20	100	100%

Distribución por tipo de pregunta (transversal a las 3 secciones):

Tipo de ítem	Cantidad	Propósito
Verdadero/Falso justificado	4	Detectar errores conceptuales frecuentes
Selección múltiple con única respuesta	4	Evaluar comprensión rápida de sintaxis/semántica
Predicción de salida (trace de código)	4	Evaluar lectura y ejecución mental de código
Problema abierto (escribir código a mano)	8	Evaluar diseño y traducción de un enunciado a código

4. Estructura y preguntas modelo

Sección I — Expresiones condicionales (30 pts)

I.1 (V/F, justifique). “En Python, elif es obligatorio después de todo if.” — Falso/Verdadero y por qué. (5 pts)

I.2 (Selección múltiple). ¿Cuál es la salida de:

```
x = -3
if x > 0:
    print("positivo")
elif x == 0:
    print("cero")
else:
    print("negativo")
```

a) positivo b) cero c) negativo d) error de indentación (5 pts)

I.3 (Predicción de salida). Dado a, b = 4, 7, indique qué imprime:

```
if a > b and b > 0:
    print("A")
elif not(a > b) or b < 0:
    print("B")
else:
    print("C")
```

(5 pts)

I.4 (Problema abierto). Escriba una función clasificar(x) que reciba un número real y retorne "negativo", "cero" o "positivo", usando if / elif / else. (5 pts)

I.5 (Problema abierto — función por partes). Defina, usando condicionales anidados, la función matemática

$$f(x) = \begin{cases} -x & x < 0 \\ x^2 & 0 \leq x \leq 2 \\ 4 & x > 2 \end{cases}$$

como una función de Python f(x). (5 pts)

I.6 (Problema abierto — validación lógica). Escriba una función es_bisiesto(año) que retorne True o False aplicando la regla: divisible entre 4, excepto los divisibles entre 100 salvo que también lo sean entre 400. (5 pts)

Sección II — Ciclos en Python (35 pts)

II.1 (V/F, justifique). “Un ciclo while True sin break interno nunca termina.” (5 pts)

II.2 (Selección múltiple). ¿Qué imprime este ciclo?

```
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
```

a) 1 2 3 4 5 b) 1 2 4 5 c) 1 2 3 d) 1 2 4 5 (se detiene en 3) (5 pts)

II.3 (Predicción de salida — ciclo anidado). Escriba la salida exacta de:

```
for i in range(3):
    for j in range(3):
        if i == j:
            print(i, j)
```

(5 pts)

II.4 (Problema abierto — sumatoria). Escriba un programa con for que calcule

$$S = \sum_{k=1}^n k^2$$

para un n dado por el usuario. (5 pts)

II.5 (Problema abierto — while con criterio de parada). Escriba, usando while, el método de bisección para aproximar una raíz de $f(x) = x^2 - 2$ en $[1, 2]$ con tolerancia 10^{-4} . (5 pts)

II.6 (Problema abierto — datos). Dada una lista de calificaciones notas, escriba un ciclo que calcule el promedio y cuente cuántas notas están por debajo de 3.0. (5 pts)

II.7 (Problema abierto — tabla). Escriba un ciclo anidado que imprima la tabla de multiplicar del 1 al 5 (5 filas $\times$ 5 columnas) con formato tabular. (5 pts)

Sección III — Funciones con def (35 pts)

III.1 (V/F, justifique). “Un parámetro con valor por defecto siempre debe declararse antes que uno sin valor por defecto.” (5 pts)

III.2 (Selección múltiple). Dada

```
def f(a, b=2, *, c):
    return a + b + c
```

¿cuál llamado es válido? a) f(1, 2, 3) b) f(1, c=3) c) f(c=3) d) f(a=1, b=2) (5 pts)

III.3 (Predicción de salida — recursión). ¿Qué retorna fib(5) si

```
def fib(n):
    if n <= 1:
        return n
    return fib(n-1) + fib(n-2)
```

? Muestre el árbol de llamadas o el razonamiento. (5 pts)

III.4 (Problema abierto — función matemática). Defina $f(x) = x^3 - 2x + 1$ como función de Python y evalúela en $x = -1, 0, 1, 2$. (5 pts)

III.5 (Problema abierto — composición). Dadas $f(x) = 2x + 1$ y $g(x) = x^2$, escriba una función `componer(f, g, x)` que retorne $f(g(x))$, usando funciones como argumentos. (5 pts)

III.6 (Problema abierto — parámetros). Escriba una función `resumen(nombre, *notas, aprobado=3.0)` que reciba un nombre y un número variable de notas, y retorne si el estudiante aprueba ($\text{promedio} \geq \text{aprobado}$). (5 pts)

III.7 (Problema abierto — sucesión recursiva). Defina por recursión la función `suma_hasta(n)` que calcule $1 + 2 + \dots + n$ sin usar ciclos, solo llamadas recursivas. (5 pts)

5. Rúbrica general de calificación (por pregunta abierta, 5 pts)

Criterio	Puntos
Sintaxis de Python correcta (dos puntos, indentación, paréntesis)	1.5
Lógica/algoritmo correcto para el caso general	2.0
Manejo correcto de casos borde (límites del dominio, condición de parada)	1.0
Claridad y organización del código o razonamiento escrito	0.5

Para V/F, selección múltiple y predicción de salida: 3 pts por la respuesta correcta + 2 pts por la justificación o el rastreo mostrado (no se otorga puntaje por respuesta sin justificar cuando esta se solicita).

6. Recomendaciones de aplicación

- Como es una prueba **escrita sin computador**, priorice que el estudiante trace el código a mano (tabla de valores de variables paso a paso) en las preguntas de predicción de salida.
- Puede rotar los valores numéricos de las preguntas modelo (III.3, II.5, I.6) entre versiones del parcial para reducir copia, manteniendo la misma estructura y puntaje.
- Si se desea una versión más corta (60 min), reduzca a 12 preguntas manteniendo la proporción 30/35/35 y elimine una pregunta abierta de cada sección.

- Los tres cuadernos ya contienen bancos de preguntas más extensos (20 c/u) de los que puede tomar variantes adicionales si se requiere un banco más grande para varias versiones del examen.